Relatório sobre Gerador de Código Intermediário

Matheus Fagundes de Oliveira
Engenharia de Computação – Campus Bagé
Universidade Federal do Pampa
E-mail: matheusoliveira.aluno@unipampa.edu.br

Thálys Lemos Correa
Engenharia de Computação – Campus Bagé
Universidade Federal do Pampa
Email: thalyscorrea.aluno@unipampa.edu.br

I. Introdução

A linguagem pode ser entendida como um sistema de símbolos ou sinais organizados em um código, cujo propósito é transmitir ideias por meio da comunicação [1]. Fundamentada em uma estrutura de gramática e vocabulário, a linguagem desempenha um papel essencial no desenvolvimento de softwares, especialmente em sua implementação. No entanto, sua aplicação no contexto da engenharia de software apresenta desafios significativos, como a ambiguidade, a incompletude e a dificuldade em obter uma visão global dos problemas envolvidos no desenvolvimento. Esses fatores aumentam substancialmente a complexidade das tarefas relacionadas à construção de software [2].

No contexto do desenvolvimento de compiladores, superar esses desafios requer o cumprimento de várias etapas até se chegar ao código de máquina final a partir de um fluxo inicial de caracteres. Essas etapas incluem a análise léxica, que gera uma lista de tokens; a análise sintática, que constrói uma árvore sintática a partir desses tokens; a análise semântica, que ajusta e valida a árvore sintática gerada; e, finalmente, a geração de código intermediário, que utiliza essa estrutura para criar uma representação intermediária adequada.

A análise léxica, etapa inicial do processo, tem como objetivo identificar trechos do código-fonte e agrupá-los em tokens. Esses tokens representam unidades significativas que servirão como entrada para a fase seguinte, a análise sintática. Durante esse processo, o analisador léxico lê os caracteres do código de entrada e organiza as informações na forma de uma tabela de tokens, essencial para o funcionamento das etapas subsequentes do compilador [3].

A análise sintática, por sua vez, é responsável por combinar os tokens gerados na análise léxica em uma estrutura hierárquica que representa a gramática do programa, geralmente na forma de uma árvore sintática. Essa árvore organiza as operações e seus operandos, permitindo uma visualização clara da hierarquia e da relação entre os componentes do código. Além disso, o analisador sintático verifica se o código segue as regras gramaticais previamente definidas, garantindo a correção estrutural antes de prosseguir com as etapas posteriores [4].

Após a análise sintática, a análise semântica entra em cena, com o objetivo de garantir que o programa não apenas obedeça à estrutura gramatical, mas também que os significados atribuídos às expressões estejam corretos. Nesta etapa, são validadas características como tipos de dados, uso correto de variáveis e coerência das operações. O analisador semântico realiza, portanto, verificações mais profundas e ajusta a árvore sintática para resolver inconsistências, assegurando uma interpretação correta do código. [3]

Com a árvore sintática validada pela análise semântica, o compilador passa para a geração de código intermediário, que consiste em criar uma representação mais próxima do código final, mas ainda independente da arquitetura da máquina-alvo. O código intermediário funciona como uma ponte entre a análise do código-fonte e a geração do código final, facilitando a otimização e a adaptação para diferentes plataformas. Essa representação intermediária organiza as instruções de forma simplificada e eficiente, permitindo que etapas subsequentes se concentrem em traduzir o programa para o formato específico da máquina-alvo. [3]

Dado esse contexto, o presente trabalho tem como objetivo principal consolidar essas etapas fundamentais do processo de compilação, por meio do desenvolvimento de um tradutor simples dirigido por sintaxe. Por se tratar de uma abordagem introdutória, e considerando que a construção de um compilador completo envolve um nível de complexidade muito maior, o foco deste projeto está na síntese do código, culminando na geração de código intermediário e, posteriormente, na tradução para assembly do MiniMIPS de 8 bits, como será detalhado nos próximos tópicos.

II. Objetivos

Com base nos conceitos apresentados na introdução, o projeto prático da disciplina Projeto Integrador III, ministrada no segundo semestre de 2024, tem como principal objetivo o desenvolvimento de um tradutor simples dirigido por sintaxe. Esse projeto é fundamentado nos conceitos abordados nos capítulos 2, 7 e 8 do livro Compiladores: Princípios, Técnicas e Ferramentas, de Alfred V. Aho, Monica S. Lam, Ravi Sethi e Jeffrey D. Ullman [3], além de utilizar a ferramenta Ply [5], implementada em Python [6].

O tradutor será construído com base nas etapas de análise léxica e sintática, culminando na geração de código intermediário. Durante esse processo, haverá interação direta com a árvore sintática gerada pelo analisador sintático e com a tabela de símbolos criada pelo analisador léxico. Por se tratar de uma disciplina introdutória e com o objetivo de manter a complexidade

controlada, a análise semântica não será abordada neste projeto, uma vez que sua inclusão exigiria uma estrutura muito mais elaborada e detalhada. Assim, o foco estará restrito às etapas iniciais de um compilador simples.
Como etapa final, o projeto tem como objetivo traduzir o código intermediário gerado para a linguagem assembly do MiniMIPS de 8 bits. O resultado será salvo em um arquivo de texto contendo o código-fonte em linguagem assembly do MiniMIPS, proporcionando uma representação funcional e organizada do programa traduzido.

III. Materiais e Métodos

A. Planejamento do Projeto

A metodologia adotada pelo grupo foi estruturada para garantir o cumprimento dos objetivos propostos dentro do prazo estipulado e assegurar a qualidade da entrega final. É importante ressaltar que esta é a terceira e última etapa do projeto, que se baseia nos resultados obtidos nas fases anteriores. Durante as etapas iniciais, já foram desenvolvidos os analisadores léxico e sintático, possibilitando a obtenção da árvore sintática a partir de uma lista de tokens gerada com base na entrada de dados. Para garantir que o trabalho fosse realizado de forma eficiente, o grupo definiu os seguintes objetivos específicos a serem cumpridos:

- Aprofundamento nos estudos sobre tradutores e a implementação de suas fases iniciais;
- Realização de ajustes no código do analisador léxico e analisador sintático, com base nas avaliações fornecidas pela docente durante o primeiro projeto;
- Implementação de um gerador de código intermediário genérico utilizando o PLY, com base na árvore sintática gerada durante a fase anterior a este projeto;
- Implementação de um tradutor do código intermediário para a linguagem assembly do MiniMIPS de 8 bits;
- Testar e ajustar o tradutor completo;
- Sintetizar o código em um único arquivo;
- Construção do relatório final.

Inicialmente, foram propostos todos os objetivos específicos que deveriam ser atendidos para culminar na conclusão da etapa. No entanto, diferentemente da abordagem utilizada na primeira fase, em que o plano de trabalho consistiu na divisão de tarefas entre os integrantes, com cada membro sendo responsável por vários objetivos específicos, optou-se nesta etapa por uma abordagem mais colaborativa.
Durante a segunda fase do projeto, foi identificada uma grande dependência entre as tarefas a serem executadas pelo grupo, o que exigiu maior sincronia entre os membros. Diante dessa experiência, a metodologia desta etapa envolveu a concentração de todos os membros do grupo em um único objetivo específico por vez, assegurando sua conclusão antes de passar para a próxima tarefa. Apesar de aumentar o tempo dedicado a cada tarefa individualmente, essa abordagem demonstrou ser mais eficiente ao minimizar falhas de integração, garantir maior coesão no desenvolvimento do projeto e evitar a sobrecarga dos membros com atividades mais complexas.
Para viabilizar essa abordagem, foram adotadas algumas práticas de trabalho. O grupo utilizou o Visual Studio Code (VScode) com a extensão Live Share [7] e o GitHub como ferramentas de integração contínua, o que permitiu a colaboração simultânea, o compartilhamento de código, a revisão de implementações e a resolução ágil de problemas. Além disso, foram realizadas reuniões periódicas via Discord e Google Meet com o objetivo de discutir o progresso, resolver problemas e alinhar os próximos passos.
Essa metodologia colaborativa permitiu ao grupo lidar de forma eficiente com imprevistos, como a sobrecarga de atividades específicas ou a falta de tempo de algum integrante. Dessa forma, todos participaram ativamente das etapas críticas do projeto, garantindo a qualidade do trabalho final e o cumprimento dos prazos definidos.

B. Desenvolvimento das Outras Etapas

Esta seção da metodologia tem como objetivo recapitular as fases anteriores do projeto. Na primeira etapa, o grupo avaliou três ferramentas para a implementação do analisador léxico: Ply, JFlex e Flex. A escolha recaiu sobre o Ply, devido à sua flexibilidade, integração com Python, suporte a ferramentas de análise sintática, facilidade de depuração e personalização. Embora JFlex e Flex oferecessem alta performance e fossem adequados a sistemas legados, a robustez e a versatilidade do Ply o tornaram a melhor opção para o projeto.
O desenvolvimento do analisador léxico foi realizado em etapas, começando pela criação de expressões regulares para os elementos da linguagem C, como operadores e palavras reservadas. O estudo do caso QuickSort orientou a implementação, e a equipe dividiu as responsabilidades para criar o código com a biblioteca Ply. O processo envolveu a identificação de tokens e a geração de uma tabela de símbolos, além de tratar erros para garantir a precisão.
O programa foi projetado para ser simples e modular, requerendo apenas a instalação do Ply e a organização dos arquivos. Ao ser executado, o script processa automaticamente o código C de entrada, identifica tokens e gera a tabela de símbolos. Os testes realizados com o código QuickSort confirmaram a eficácia do analisador, com resultados precisos, embora tenha sido possível identificar pequenas melhorias no reconhecimento de caracteres específicos.

Na segunda etapa do projeto de construção de um compilador, o grupo concentrou-se no desenvolvimento do analisador sintático e na implementação dos parsers de entrada. Essa fase exigiu um aprofundamento nos conceitos relacionados à análise sintática, com o objetivo de compreender e superar os desafios inerentes à criação de um software voltado para essa área. Inicialmente, os estudos foram guiados pela necessidade de atender às avaliações fornecidas pela docente, que incluíam a verificação do suporte correto ao conjunto dos números naturais e a medição do tempo investido em cada atividade para uma análise comparativa posterior.

O processo começou com uma revisão sobre sistemas de reescrita, gramáticas livres de contexto e estruturas hierárquicas de linguagens, elementos fundamentais para a implementação de um analisador sintático robusto. Após uma fase inicial de experimentação, cada membro do grupo passou a implementar individualmente o analisador sintático, compartilhando dificuldades e soluções em reuniões periódicas. Um dos problemas mais críticos identificados foi a inconsistência no comportamento do programa entre diferentes ambientes de desenvolvimento.

O código foi inicialmente implementado no Google Colab, utilizando arquivos salvos no Google Drive como entrada. No entanto, surgiram erros recorrentes, incluindo atribuições incorretas e a leitura inadequada de tokens. Esses problemas inviabilizavam o funcionamento correto do analisador. Quando testado na IDE VScode para Linux, o mesmo código funcionava corretamente, sem apresentar os erros observados no Colab. Essa solução não apenas eliminou os problemas de inconsistência, mas também promoveu um ambiente de trabalho integrado, onde cada membro do grupo podia contribuir ativamente para a construção do analisador sintático. Essa experiência destacou a importância de escolher ferramentas adequadas e adaptáveis ao contexto do projeto. Ao final dessa etapa, o grupo conseguiu implementar um analisador sintático funcional, capaz de processar corretamente as entradas previstas no escopo do projeto.

Após a finalização da construção do código e a resolução de problemas de compilação, iniciou-se a etapa de testes e correções. Essa fase foi essencial para verificar o funcionamento do programa e identificar ajustes necessários. O processo envolveu a criação de pequenos programas em C, que foram utilizados como entrada para o analisador léxico. A análise desses programas permitiu validar o comportamento do código e identificar áreas específicas que precisavam de refinamento.

C. Implementação da Geração do Código Intermediário

Diferentemente dos analisadores léxico e sintático, que se constituem em seções do código com atribuições, especificações e, principalmente, conjuntos de funções, o gerador de código intermediário foi implementado como uma classe *CodeGenerator* que desempenha o papel de traduzir a árvore sintática gerada pelo analisador sintático em uma sequência de instruções intermediárias. Esse conjunto de instruções é armazenado em uma lista *instructions*, onde cada instrução é representada como uma tupla contendo quatro elementos: o operador, os dois operandos (ou None quando não é utilizado) e o destino ou resultado da operação. Por exemplo, uma instrução de operação binária como `x = 2 + 3` resultaria na instrução `('+','2','3','x')`, enquanto instruções de atribuição como `x = 4` seriam armazenadas como `('=','4', None, 'x')`.

O método base para a geração de código é o *emit*, que adiciona uma nova instrução à lista instructions. Esse método permite incluir qualquer tipo de instrução, como operações aritméticas, atribuições, saltos condicionais e rótulos. Por exemplo, ao emitir uma instrução para somar duas variáveis a e b e armazenar o resultado em um temporário t1, o método seria chamado como *self.emit('+', 'a', 'b', 't1')*. A geração de código para os nós da árvore sintática de atribuição ocorre em duas etapas. Primeiramente, o gerador avalia a expressão do lado direito da atribuição, chamando recursivamente o método *generate*. Esse processo retorna o conjunto de instruções geradas para a expressão, juntamente com o resultado final (geralmente uma variável ou valor). Em seguida, uma instrução de atribuição é gerada usando o *emit*, que armazena o resultado na variável de destino. Por exemplo, para o código `x = a + b`, o gerador criaria instruções intermediárias para calcular a soma e, em seguida, atribuir o resultado a x.

A classe também gerencia variáveis temporárias e rótulos de controle de fluxo por meio de contadores: *temp_count* e *label_count*, que geram, respectivamente, identificadores para variáveis temporárias como t1, t2, etc., e criam rótulos identificados como L1, L2, etc. Esses recursos servem para funções auxiliares que retornam novas variáveis temporárias e novos rótulos, os quais são muito importantes para as operações do programa. Por exemplo, para as operações binárias, como soma, subtração, multiplicação e divisão, a representação gerada pelo nó é *('binop', op, left, right)*, onde op é o operador e left e right são expressões nos lados esquerdo e direito que serão processadas por meio da chamada recursiva do método generate, e os resultados serão armazenados em variáveis temporárias. Por exemplo, para a expressão `a + b * c`, o gerador criaria instruções para calcular `b * c` e, em seguida, somar o resultado com `a`, usando variáveis temporárias.

No caso dos laços *for*, eles são representados pelo nó *('for', init, cond, increment, body)*, que contém a inicialização, a condição, o incremento e o corpo do laço. O gerador processa esses elementos em sequência: primeiramente, gera instruções para a inicialização; depois, cria um rótulo para o início do laço e uma instrução de salto condicional para verificar a condição. Se a condição for falsa, o controle é direcionado para o rótulo do final do laço. Em seguida, as instruções do corpo do laço são geradas, seguidas pelo incremento, e o controle retorna ao rótulo do início do laço. Finalmente, um rótulo é inserido no final do laço para marcar o ponto de saída. Essa estrutura assegura que o fluxo do laço seja corretamente representado no código intermediário, permitindo que os saltos sejam realizados para os endereços apropriados por meio dos rótulos.

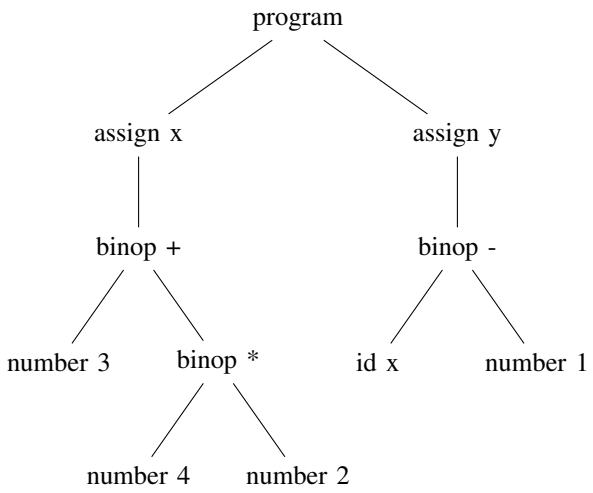
Estruturas de controle, como o switch-case, são representadas pelo nó *('switch', expr, cases)*, onde *expr* é a expressão de controle e *cases* é a lista de casos. Para cada caso, o gerador compara o valor de *expr* com o valor do caso utilizando uma instrução condicional. Se a condição for verdadeira, o controle salta para o corpo do caso correspondente. Um rótulo final é utilizado para marcar o fim do bloco switch, e os comandos *break* geram saltos para esse rótulo. A implementação também contempla o caso padrão, que é executado caso nenhum dos valores dos casos coincida com a expressão. O comando *break* é tratado de maneira simples, com um salto para o rótulo armazenado no topo da pilha de rótulos. Essa pilha é fundamental para gerenciar os rótulos de saída de estruturas como *for* e *switch*, garantindo que o *break* funcione corretamente em níveis de aninhamento diferentes. Nós simples, como números (*('number', value)*) e identificadores (*('id', name)*), são processados diretamente, retornando seu valor ou nome sem a necessidade de gerar novas instruções. Isso simplifica a avaliação de expressões compostas, já que valores básicos são tratados como operandos prontos para uso em operações mais complexas.
Por fim, a classe oferece funcionalidades para exibir e exportar as instruções geradas. O método *display* exibe as instruções intermediárias na tela, facilitando a depuração durante o desenvolvimento. O método *export_to_assembly*, por sua vez, converte as instruções intermediárias para um formato específico de Assembly (neste caso, MiniMIPS) e as salva em um arquivo. Ele mapeia cada instrução intermediária para sua correspondente em Assembly. Por exemplo, uma instrução de soma, como *('+', 'a', 'b', 't1')*, seria convertida para a instrução `add $t1, a, b` em MiniMIPS.

IV. Resultados e Discussões

Após a validação dos códigos individuais, que estavam funcionando conforme o esperado, foi realizada a integração dos analisadores léxico, sintático e do gerador de código intermediário em um único programa. Além disso, foi incorporada uma seção específica para o armazenamento e leitura de códigos em linguagem C. Desenvolvida em Python, essa seção possibilitou a inclusão direta dos programas em C no código, otimizando o processo de depuração e manipulação pelo tradutor.
O primeiro teste realizado envolveu duas expressões matemáticas simples:

```
x = 3 + 4 * 2;
y = x - 1;
```

A representação da árvore sintática gerada pelo analisador léxico foi elaborada utilizando o pacote TikZ [8] na ferramenta Overleaf, conforme apresentado a seguir:



Apresenta-se, abaixo, o código intermediário à esquerda e o código em Assembly do MiniMIPS de 8 bits à direita. Vale destacar que, embora diversas instruções não estejam presentes no conjunto original do MiniMIPS, utilizamos suas representações para facilitar a compreensão, como é o caso da função mul:

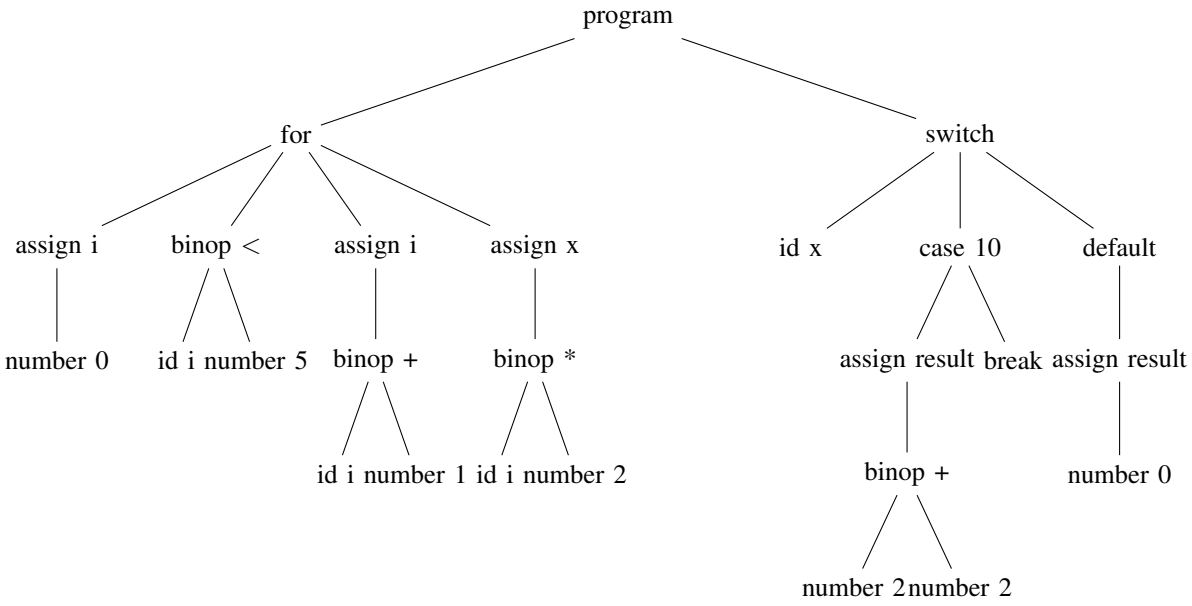
```
0: ('*', 4, 2, 't1'))
1: ('+', 3, 't1', 't2')
2: ('=', 't2', None, 'x')
3: ('-', 'x', 1, 't3')
4: ('=', 't3', None, 'y')
```

```
mul $t1, $4, $2
add $t2, $3, $t1
li $x, t2
sub $t3, $x, $1
li $y, t3
```

Para um segundo teste, foi utilizado um programa mais complexo e mais elaborado, envolvendo estruturas de controle como `for` e `switch`, representado logo abaixo:

```
for (i = 0; i < 5; i = i + 1){
  // Loop para realizar ações
  x = i * 2;
}
switch (x){
  case 10: result = 2 + 2; break;
  default: result = 0;
}
```

A seguir, apresenta-se a árvore sintática gerada pelo código utilizando o pacote TikZ:



Abaixo está o código intermediário gerado à esquerda e o código em Assembly do MiniMIPS de 8 bits à direita:

```
0: ('=', 0, None, 'i')
1: ('label', 'L1', None, None)
2: ('<', 'i', 5, 't1')
3: ('if_false', 't1', None, 'L2')
4: ('*', 'i', 2, 't2')
5: ('=', 't2', None, 'x')
6: ('+', 'i', 1, 't3')
7: ('=', 't3', None, 'i')
8: ('goto', 'L1', None, None)
9: ('label', 'L2', None, None)
10: ('if_eq', 'x', 10, 'L4')
11: ('goto', 'L3', None, None)
12: ('label', 'L4', None, None)
13: ('+', 2, 2, 't4')
14: ('=', 't4', None, 'result')
15: ('goto', 'L3', None, None)
16: ('=', 0, None, 'result')
17: ('label', 'L3', None, None)
```

```
move $i, 0
L1:
beqz $t1, L2
mul $t2, i, 2
move $x, $t2
add $t3, i, 1
move $i, $t3
j L1
L2:
j L3
L4:
add $t4, 2, 2
move $result, $t4
j L3
move $result, 0
L3:
```

Após as implementações, um relatório foi produzido, documentando os tempos de execução registrados durante as atividades. Os tempos de execução foram cronometrados considerando o esforço coletivo do grupo para cada tarefa, em vez de medições individuais. Essa abordagem permitiu uma avaliação abrangente da eficiência operacional. A Tabela abaixo apresenta os tempos

coletados e suas respectivas estimativas.

Atividades realizadas pelo grupo	Tempo estimado	Tempo registrado
Aprofundamento nos estudos	2 horas	3 horas
Ajustes nos analisadores	1 hora	2 horas
Implementação de gerador intermediário	3 horas	5 horas
Testes e correções	1 hora	2 horas
Implementação do tradutor para MiniMIPS	1 hora	2 horas
Elaboração do relatório	6 horas	8 horas

Os testes realizados confirmaram o funcionamento adequado do tradutor simples dirigido por sintaxe. Os tempos registrados evidenciam a eficiência do grupo em cumprir os prazos estabelecidos, ressaltando a importância do planejamento e da colaboração ao longo do projeto. Esses resultados não apenas validam a funcionalidade do sistema, mas também destacam sua aplicabilidade na análise de códigos reais, consolidando os objetivos inicialmente propostos.

V. Considerações Finais

O desenvolvimento do tradutor simples dirigido por sintaxe foi concluído com êxito, atendendo aos objetivos estabelecidos no início do projeto. A implementação e os testes realizados permitiram a validação dos conceitos adquiridos ao longo da disciplina, com foco nas análises léxica e sintática, além da geração de represenstações intermediárias de linguagens formais. O sistema foi projetado para interpretar programas escritos em C, identificando e processando suas estruturas por meio de um fluxo de caracteres, que é transformado primeiro em uma lista de tokens, depois em uma árvore sintática, seguida de uma representação intermediária de código, e finalmente convertido em código assembly para o MiniMIPS de 8 bits. Isso demonstrou a viabilidade de construir um compilador simplificado.

Durante o desenvolvimento, a equipe enfrentou desafios, como a adaptação do código às especificidades da linguagem C, inconsistências no comportamento do código e a falta de experiência na criação de tradutores. Esses obstáculos foram superados por meio de ajustes nas expressões regulares, no fluxo de entrada de dados, no aprofundamento dos estudos sobre compiladores, na mudança de abordagem no planejamento de tarefas e na busca contínua por erros, resultando em uma solução funcional e eficiente.

Ao concluir o projeto, não apenas foram cumpridas as metas, mas também foi possível alcançar uma compreensão mais profunda sobre a complexidade de criar um software de tradução completamente do zero. A experiência proporcionou um valioso aprendizado, especialmente em áreas como depuração, integração de componentes e adaptação do código a diferentes cenários. O sistema desenvolvido serve como uma base sólida para futuras melhorias, incluindo a implementação de novos recursos e a expansão para suportar outras linguagens de programação.

Referências

[1] E. d. C. VIOTTI, "Introdução aos estudos linguísticos." Florianópolis: Universidade Federal de Santa Catarina, 2008., acesso em: 10 de novembro de 2024.

[2] K. M. e. a. ANNERVAZ, "Natural language requirements quality analysis based on business domain models," 28th IEEE/ACM International Conference on Automated Software Engineering (ASE). IEEE, 2013. p. 676-681, 2013, acesso em: 12 de novembro de 2024.

[3] M. S. S. R. U. J. D. AHO, Alfred V.; LAM, "Compiladores: Princípios, técnicas e ferramentas." Pearson Universidades. 2ª edição, 2007. 648p., acesso em: 13 de dezembro de 2024.

[4] C. R. e. C. J. M. M. BARBOSA, C. R. S. C. de; FARIA, "Análise de ferramentas de compiladores em ambientes virtualizados." Revista Brasileira de Informática na Educação, [S. l.], v. 29, p. 1262–1290, 2021. DOI: 10.5753/rbie.2021.2302. Disponível em: https://journals-sol.sbc.org.br/index.php/rbie/article/view/2302., acesso em: 16 de novembro de 2024.

[5] D. BEAZLEY, "Ply: Python lex-yacc," 2016, acesso em: 16 de novembro de 2024. [Online]. Available: https://www.dabeaz.com/ply/ply.html

[6] P. S. FOUNDATION., "Python language reference, version 3.9." https://docs.python.org/3/, 2024, acesso em: 25 de setembro de 2024.

[7] MICROSOFT., "Live share." Disponível em: https://github.com/microsoft/live-share., acesso em: 10 de novembro de 2024.

[8] OVERLEAF., "Tikz package." Disponível em: https://www.overleaf.com/learn/latex/TikZ$_package., acessoem$: 20$dedezembrode$2024.